
When Does Video Pretraining Help? Improving Embodied Navigation with Next-Frame Prediction

Batu El
Stanford University
batuel@stanford.edu

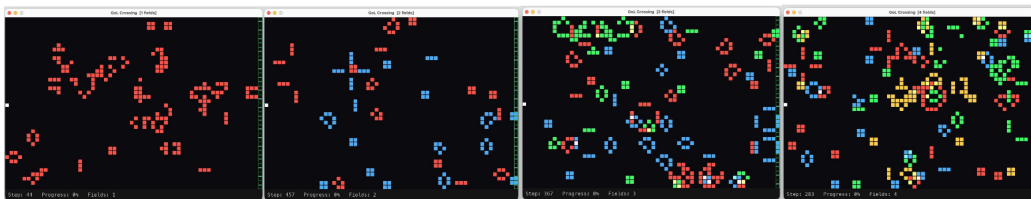


Figure 1: *Example game environment setup showing the Frame Crossing Game at increasing complexity levels (1–4 fields). The agent (white cell, left edge) must navigate across the 96×96 grid to the right edge while avoiding live cells, whose evolution follows Conway’s Game of Life rules. Each additional field introduces an independent population of cells (shown in distinct colors), increasing environment complexity and obstacle density.*

Abstract

Video pretraining can teach a model to predict future states with high fidelity, yet how this knowledge translates into better performance in downstream embodied tasks is not well understood. Some studies report substantial transfer benefits, while others demonstrate that training from scratch achieves comparable performance. We isolate this question in a controlled benchmark built on Conway’s Game of Life, where transition dynamics are exactly known and the pretraining and RL environments share identical dynamics, neutralizing the domain gap that confounds real-world transfer studies. In our *Frame Crossing Game*, an agent must navigate a 96×96 grid while avoiding live cells evolving under Game of Life rules. We pre-train a neural network via next-frame prediction, achieving near-perfect accuracy, and compare pretrained RL agents against agents trained from scratch across environment complexities (1–4 fields) and model capacities (1–4 layers). We have two main findings: (1) The benefit of pretraining depends on environment complexity, which is precisely controlled in our setup. In easy environments, pretraining accelerates convergence but both approaches reach the same asymptote; in hard environments, all agents struggle regardless of initialization; at intermediate difficulty, pretrained agents converge faster and to a strictly higher performance level. (2) Smaller models benefit more from pretraining than larger ones. Larger models achieve lower pretraining loss, as expected, but better prediction accuracy does not translate into better RL initialization. Together these results indicate that pretraining is most valuable when the environment is complex enough that anticipating future states matters but not so complex that planning becomes intractable, and when model capacity is limited enough that a warm start cannot be easily recovered from scratch. <https://github.com/batu-el/when-vpt>

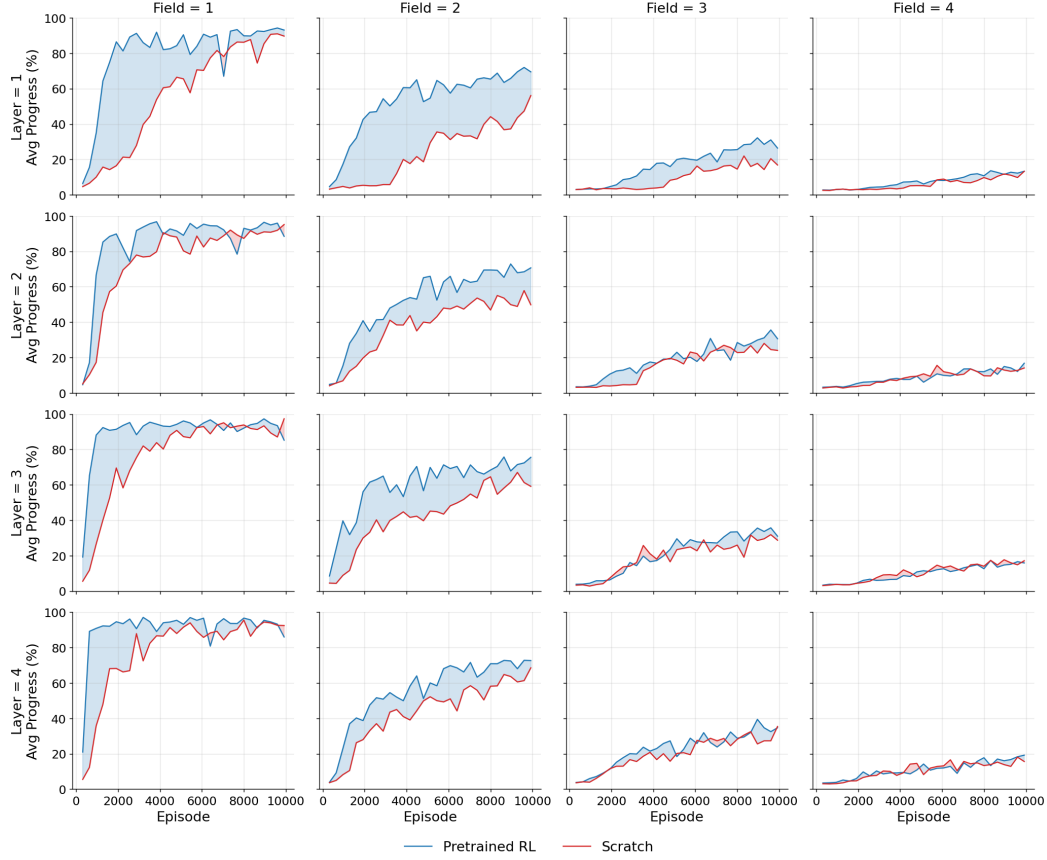


Figure 2: *Pretrained vs. scratch RL training curves across environment complexities (columns: 1–4 fields) and model capacities (rows: 1–4 layers).* Progress is measured as the percentage of columns crossed (reaching column 48 of 96 corresponds to 50%). The x -axis denotes cumulative training episodes, and curves are smoothed with a moving average for clarity. Blue curves show agents initialized from video-pretrained weights; red curves show agents trained from scratch. Shaded regions highlight the gap between the two. In easy settings (Field = 1), pretraining accelerates convergence but both approaches reach similar asymptotic performance. At intermediate difficulty (Field = 2), pretrained agents converge faster and to a higher level of progress. In the hardest settings (Field = 4), all agents struggle regardless of initialization, and the pretraining advantage largely vanishes. Across all difficulty levels, the pretraining advantage is largest for smaller models and shrinks as capacity increases.

1 Introduction

In the universe, physical laws govern how the current state transitions into the next over time. At the microscopic scale, the electromagnetic forces determine how a charged particle curves through a magnetic field. At the macroscopic scale, gravitational attraction shapes a planet’s orbit around its star. Our perception of the world captures these transition dynamics at the resolution of visible objects: an apple falls from a tree, strikes the ground, then bounces and rolls away. By learning from spatiotemporal signals, humans are able to anticipate future states, which allows them to perform remarkable actions such as catching a ball by predicting its trajectory or dodging a punch by inferring the path of an incoming strike. This capacity for predictive world modeling appears central to choosing effective actions.

Background. Recent advances in video generation have renewed interest in whether generative models can internalize physical dynamics [Liu et al., 2024, Ren et al., 2025, 2026]. Parallel work on world models has shown that agents can learn effective behaviors by predicting future states in latent space [Hafner et al., 2020, 2025], and a growing body of evidence suggests that models pretrained on video and image data acquire representations useful for downstream control, particularly in robot manipulation [Nair et al., 2022, Radosavovic et al., 2022, Xiao et al., 2022, Parisi et al., 2022, Brohan et al., 2023, Wu et al., 2023]. Yet the evidence is mixed: other work shows that simple baselines trained from scratch with data augmentation can match or exceed pretrained representations across multiple task domains [Hansen et al., 2023], that pretrained visual features fail to improve sample efficiency in model-based RL [Schneider et al., 2025], and that dataset choice matters more than the pretraining algorithm itself [Dasari et al., 2023].

Motivation. These conflicting findings suggest that the benefit of pretraining is not universal but depends on factors that are difficult to disentangle in real-world experiments, where dynamics complexity, partial observability, and distribution shift are confounded. We design a synthetic setting that neutralizes these confounders: our pretraining videos and RL environment share identical dynamics, visual format, and state representation, eliminating domain gap entirely, and the agent observes the full grid state, removing partial observability. With these factors controlled, we isolate two variables (environment complexity and model capacity) and study how they influence the gap between pretrained and from-scratch agents.

Environment. To isolate and study this question in a controlled setting, we introduce a synthetic benchmark built on Conway’s Game of Life (GoL) [Gardner, 1970], which is a deterministic cellular automaton with local update rules and known transition dynamics. We define the *Frame Crossing Game*, in which an agent must navigate across a 96×96 grid while avoiding live cells whose evolution is governed by GoL rules. This environment allows precise control over complexity (via the number of simultaneous GoL fields) while keeping the underlying dynamics tractable and exactly known.

Learning. We train our models via self-supervised next-frame prediction on synthetic video sequences generated by this process and evaluate whether the learned representations transfer to the downstream reinforcement learning task. Our pretraining stage achieves near-perfect prediction accuracy, confirming that the model reliably internalizes the local update rules. We then compare RL agents initialized from pretrained weights against agents trained from scratch across a sweep of environment complexities (1-4 simultaneous GoL fields) and model capacities (1-4 layers).

Finding 1: Pretraining benefit depends on environment complexity. In easy environments (one GoL field), pretraining accelerates convergence but both approaches reach the same asymptotic performance. In hard environments (four fields), all agents struggle regardless of initialization, and the pretraining advantage largely vanishes. The most striking difference emerges at intermediate difficulty (two to three fields), where pretrained agents converge faster and to a strictly higher performance level.

Finding 2: Smaller models benefit more from pretraining than larger ones. Larger models achieve lower pretraining loss, as expected (Figure 4), and converge to higher asymptotic performance overall (Figure 5). However, the marginal contribution of pretraining to episode progress diminishes as model capacity increases.

Together, these results indicate that pretraining is most valuable when the environment is complex enough that anticipating future states matters but not so complex that planning becomes intractable, and when model capacity is limited enough that a warm start cannot be easily recovered from scratch. These trends are consistent across two reward functions and different weight decay values, suggesting that the findings are not artifacts of specific hyperparameter choices (see Appendix C).

2 Background

2.1 When Pretraining Helps

Several lines of work report positive transfer from visual pretraining to control. Self-supervised objectives such as masked image modeling [Xiao et al., 2022, Radosavovic et al., 2022] produce representations that improve sample efficiency in downstream manipulation tasks, and Parisi et al. [2022] find that even generic pretrained vision models transfer surprisingly well. A complementary approach learns a predictive world model and trains an RL agent within its predictions [Hafner et al.,

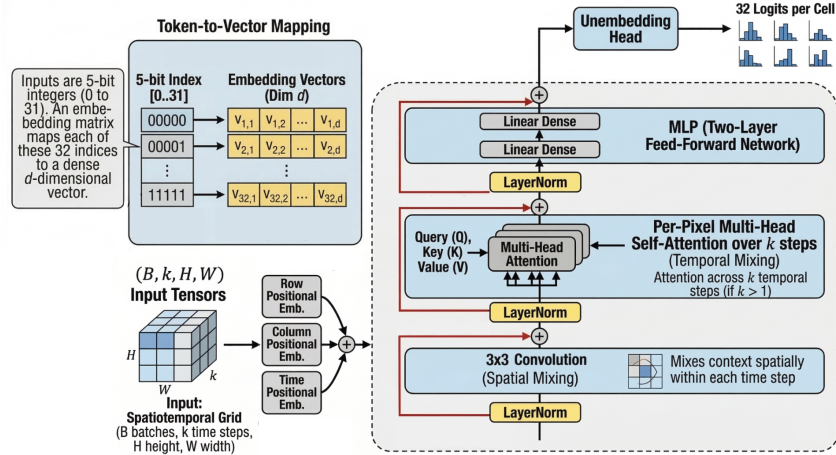


Figure 3: *Model architecture.* The input is a tensor of shape (B, k, H, W) representing k binary grid frames. Each cell is encoded as a 5-bit token (vocabulary size $V=32$) and embedded to dimension d , then summed with learned row, column, and time positional embeddings. Each layer applies three operations with residual connections: (1) a 3×3 depthwise convolution for spatial mixing within each time step, (2) per-pixel multi-head self-attention over the k temporal steps for temporal mixing, and (3) a two-layer feed-forward network (MLP). LayerNorm is applied before each operation. An unembedding head maps the final representations to 32 logits per cell, producing a next-frame prediction. Model capacity is varied by stacking 1–4 such layers. *Game of Life with history-order $M = 1$ is a Markov process whose next state is fully determined by the current state, so we set $k = 1$ in all reported experiments; in this case the temporal attention reduces to a no-op, but we retain it in the architecture to emphasize that the approach extends naturally to $M \geq 1$ and to general-purpose video pretraining where conditioning on multiple past frames is necessary.*

2020, 2025]. Video data has also been leveraged directly: VPT [Baker et al., 2022] pseudo-labels Minecraft footage via an inverse dynamics model, while other work pretrains on large-scale, out-of-domain video and transfers the learned representations to robotic manipulation [Wu et al., 2023]. Similarly, large-scale video generation models have raised the prospect that generative pretraining can internalize physical dynamics [Liu et al., 2024, Ren et al., 2025, 2026].

2.2 When Pretraining Fails

Other work challenges the assumption that pretraining reliably improves embodied control. Hansen et al. [2023] revisit a simple learning-from-scratch baseline with data augmentation and a shallow ConvNet, and find it surprisingly competitive with recent pretrained representations across multiple task domains, attributing the gap to a significant domain shift between pretraining data and downstream control benchmarks. Schneider et al. [2025] extend this analysis to model-based RL, showing that pretrained visual representations fail to improve sample efficiency in that setting. Dasari et al. [2023] further demonstrate that dataset choice matters more than algorithm design: traditional vision datasets like ImageNet are surprisingly competitive with specialized robotics pretraining, and the image distribution matters more than dataset size.

3 Setup

Environment. We build on Conway’s Game of Life (GoL) [Gardner, 1970], a deterministic cellular automaton on a binary grid with rule B3/S23 (birth at exactly 3 live neighbors; survival at 2 or 3). To scale complexity, we run $K \in \{1, 2, 3, 4\}$ independent GoL fields simultaneously on a 96×96 grid; a cell is dangerous if *any* field is alive there. Because the standard rule is first-order Markov and local (3×3 neighborhood), we also introduce a history-order extension ($M \geq 1$) that sums neighbor counts across M frames. However, due to time and resource constraints, for all results we report, we have $M = 1$.

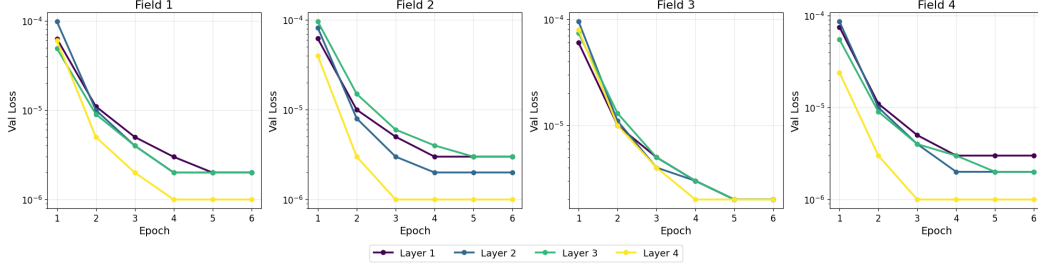


Figure 4: *Pretraining validation loss across environment complexities and model sizes.* Each subplot corresponds to a different number of simultaneous GoL fields (1–4), and each curve to a different model depth (1–4 layers). The y -axis shows per-cell cross-entropy on a held-out validation set (log scale). Larger models consistently achieve lower final loss across all field counts, with the 4-layer model reaching $\sim 10^{-6}$ by epoch 4 in every setting. All configurations converge within 6 epochs.

Task. In the *Frame Crossing Game*, the agent starts at the left edge and must reach the right edge of the grid while avoiding all live cells, which evolve under GoL dynamics. Episodes last at most 150 steps and terminate on collision or success.

Model. The world model is a spatial-temporal transformer operating on (B, k, H, W) input tensors. Each cell is encoded as a 5-bit token (4 bits for 4 GoL fields + 1 bit for player presence; vocabulary $V = 2^5 = 32$), embedded to dimension d with learned row, column, and time positional embeddings. Each of L blocks applies, in sequence: (i) a 3×3 depthwise convolution for spatial mixing, (ii) per-pixel multi-head self-attention over the k time steps, and (iii) a two-layer MLP, all with residual connections and LayerNorm. In all reported experiments we set $k = 1$, since the standard B3/S23 rule is first-order Markov: the next frame is fully determined by the current frame, making temporal context unnecessary. An unembedding head projects the final time-step features to 32 logits per cell. *During RL, no separate policy head is added: for each of the five actions {stay, up, down, left, right}, the log-probability that the player token occupies the corresponding target cell serves as the action logit.*

Pretraining. We generate 500 videos of 300 frames each across initial densities $\{0.10, 0.15, \dots, 0.50\}$, split 90/10 into training and validation sets. The model is trained for 6 epochs with batch size 256 and a peak learning rate of 10^{-3} (linear warmup over 200 steps, cosine decay to 1%). The loss is per-cell cross-entropy over the 32-class vocabulary, $\mathcal{L}_{\text{CE}}$.

Reinforcement learning. We train for 10,000 episodes in batches of 16, with learning rate 10^{-4} when fine-tuning from pretrained weights and 3×10^{-4} when training from scratch. The RL loss combines three terms:

$$\mathcal{L}_{\text{RL}} = \mathcal{L}_{\text{REINFORCE}} + \lambda_{\text{CE}} \mathcal{L}_{\text{CE}} - \lambda_{\text{ent}} \mathcal{H}(\pi), \quad (1)$$

where $\lambda_{\text{CE}} = 0.5$ and $\lambda_{\text{ent}} = 0.01$. Rewards comprise a per-step penalty (-0.1), a milestone bonus for each new rightmost column reached, and a win bonus. Under reward scheme R_1 , the milestone bonus is a flat $+3.0$ per column and the win bonus is $+50.0$. Under R_2 , the milestone bonus is *progressive*: reaching column c yields $+0.6c$, and the win bonus is $+1,000$. The REINFORCE [Williams, 1992] term uses reward-to-go $G_t = \sum_{t'=t}^T \gamma^{t'-t} r_{t'}$ with $\gamma = 0.99$. An exponential moving average baseline $b \leftarrow 0.99b + 0.01 \sum_t r_t$ is subtracted and the resulting advantages $A_t = G_t - b$ are standardised within each episode, giving

$$\mathcal{L}_{\text{REINFORCE}} = -\frac{1}{T} \sum_{t=1}^T \log \pi_{\theta}(a_t | s_t) \hat{A}_t. \quad (2)$$

The auxiliary $\mathcal{L}_{\text{CE}}$ reuses the pretraining objective to preserve world-modelling capability, and $\mathcal{H}(\pi)$ discourages premature policy collapse. Gradients are clipped to unit norm.

Training Hyperparameters. Both stages use AdamW [Loshchilov and Hutter, 2019] with weight decay 10^{-4} , gradient clipping at max norm 1.0, and mixed-precision (bfloat16).

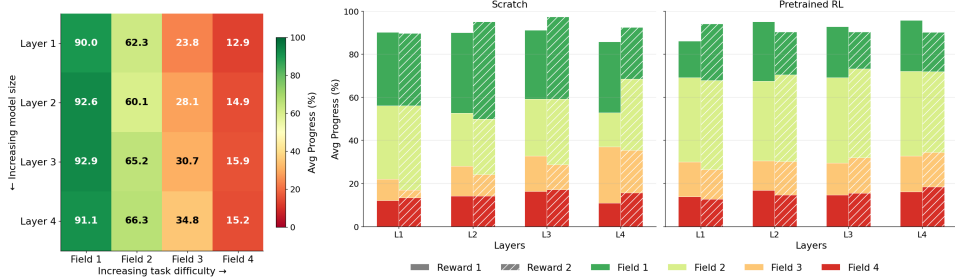


Figure 5: *Left: Asymptotic RL performance across environment complexity and model capacity.* Heatmap of average progress (%) after 10,000 training episodes for each combination of model depth (1–4 layers) and field count (1–4 fields). Performance degrades sharply with increasing field count—from ~90% at Field 1 to ~13–15% at Field 4, while increasing model capacity yields only modest gains (23.8% → 34.8% at Field 3). *Right: RL performance comparison between two rewards.* Stacked bar chart comparing average progress under reward schemes R_1 (solid) and R_2 (hatched) for scratch-trained (left panel) and pretrained (right panel) agents. Colors indicate field count.

4 Experiments

We evaluate four aspects of our pipeline: (1) pretraining loss curves (Section 4.1), (2) how RL performance varies across environment complexity and model capacity (Section 4.2), (3) the effect of reward function design on learning dynamics (Section 4.3), and (4) whether video-pretrained initialization leads to a measurable advantage over training from scratch (Section 4.4).

4.1 Pretraining

Figure 4 shows validation loss curves for all combinations of field count and model depth. Every configuration converges to near-zero cross-entropy within six epochs, confirming that the model reliably learns the GoL update rules. Larger models reach the lowest final loss ($\sim 10^{-6}$), while smaller models plateau slightly higher but still at extremely low values.

4.2 Difficulty Levels and Model Sizes

Figure 5 (left) summarizes asymptotic RL performance across all complexity–capacity combinations. The dominant factor is environment difficulty: average progress falls from ~90% at Field 1 to ~13–15% at Field 4, regardless of model size. Increasing depth yields modest but consistent improvements that are most apparent at intermediate difficulty. For example, progress at Field 3 rises from 23.8% with a single layer to 34.8% with four layers, but at Field 4 all models converge to a narrow ~2-percentage-point band, suggesting that the task exceeds what additional capacity alone can address.

4.3 Reinforcement Learning Rewards

Figure 5 (right) compares the two reward schemes. R_1 and R_2 produce broadly similar performance profiles across all conditions, confirming that our findings are robust to reward shaping. Minor differences emerge at intermediate difficulty—for example, at Layer 4 with Field 2, R_2 yields 68.5% progress compared to 52.8% for R_1 in the pretrained setting, but the overall ranking of conditions is preserved.

4.4 Pretrained vs. Scratch Comparison

Figure 2 presents RL training curves for pretrained (blue) and scratch (red) agents across all 16 combinations of field count and model depth, using reward scheme R_2 and weight decay 10^{-4} . Results under R_1 and alternative weight decay values are qualitatively similar (see Appendix C).

Finding 1: Pretraining benefit depends on environment complexity. At Field 1, both approaches reach ~90–95% progress, but pretrained agents converge faster. At Field 2–3, pretrained agents

converge faster and reach a strictly higher asymptote, as their knowledge of cell dynamics provides an anticipatory edge the scratch agent cannot fully recover. At Field 4, both conditions struggle to exceed 15–20%. This may be because as the viable paths become rare, the bottleneck shifts from dynamics prediction to combinatorial planning, which pretraining alone cannot resolve.

Finding 2: Smaller models benefit more from pretraining. Across all difficulty levels, the pretraining advantage is largest for smaller models and shrinks as depth increases. This suggests that larger models may be able to recover from scratch the representations that pretraining provides.

5 Discussion

5.1 Limitations

Convergence at intermediate difficulty. At intermediate complexity (Fields 2–3), pretrained agents converge faster and to a higher asymptote within our 10,000-episode budget. An open question is whether scratch-trained agents would eventually close this gap given substantially more training. Our experiments do not rule out this possibility, and longer training runs would be needed to determine whether the advantage is transient or persistent.

Temporal context. Although our architecture supports multi-frame conditioning ($k > 1$), the standard Game of Life rule (B3/S23) is first-order Markov, so all reported experiments use $k = 1$. In this setting, next-frame prediction is effectively single-image prediction rather than true video prediction with temporal dependencies. We designed the history-order extension ($M \geq 1$) precisely to introduce genuine temporal structure, but time and resource constraints prevented us from exploring it within the scope of this project.

Synthetic environment. While our benchmark offers precise control and eliminates confounders such as domain gap and partial observability, it remains a synthetic grid world. Real-world control tasks involve continuous state spaces, noisy observations, and complex visual inputs that may interact with pretraining in ways our setup does not capture. Validating these findings on realistic robotic or navigation benchmarks is an important next step.

5.2 Future Work

Our experimental framework was designed with a broader set of research questions in mind. While this paper addresses the core question of when video pretraining helps RL, several natural extensions remain.

Multi-frame history. Exploring the history-order extension ($M > 1$) would make the environment non-Markovian in any single frame, testing whether true temporal video pretraining provides a stronger advantage.

Rule generalization. Pretraining on diverse Game of Life variants and evaluating on unseen rules would test whether learned representations generalize beyond the specific dynamics seen during training.

Test-time adaptation. An agent deployed under novel dynamics could use test-time training to update its world model on the fly, enabling continual adaptation without retraining.

Real-world transfer. Extending this analysis to realistic benchmarks such as visual navigation would test whether the complexity- and capacity-dependent trends hold in more realistic scenarios.

References

- Bowen Baker, Ilge Akkaya, Peter Zhokhov, Joost Huizinga, Jie Tang, Adrien Ecoffet, Brandon Houghton, Raul Sampedro, and Jeff Clune. Video pretraining (vpt): Learning to act by watching unlabeled online videos, 2022. URL <https://arxiv.org/abs/2206.11795>.
- Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, Pete Florence, Chuyuan Fu, Montse Gonzalez Arenas, Keerthana Gopalakrishnan, Kehang Han, Karol Hausman, Alexander Herzog, Jasmine Hsu, Brian Ichter, Alex Irpan, Nikhil Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang,

- Isabel Leal, Lisa Lee, Tsang-Wei Edward Lee, Sergey Levine, Yao Lu, Henryk Michalewski, Igor Mordatch, Karl Pertsch, Kanishka Rao, Krista Reymann, Michael Ryoo, Grecia Salazar, Pannag Sanketi, Pierre Sermanet, Jaspiar Singh, Anikait Singh, Radu Soricut, Huong Tran, Vincent Vanhoucke, Quan Vuong, Ayzaan Wahid, Stefan Welker, Paul Wohlhart, Jialin Wu, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Tianhe Yu, and Brianna Zitkovich. Rt-2: Vision-language-action models transfer web knowledge to robotic control, 2023. URL <https://arxiv.org/abs/2307.15818>.
- Sudeep Dasari, Mohan Kumar Srirama, Unnat Jain, and Abhinav Gupta. An unbiased look at datasets for visuo-motor pre-training, 2023. URL <https://arxiv.org/abs/2310.09289>.
- Martin Gardner. Mathematical games: The fantastic combinations of john conway’s new solitaire game “life”. *Scientific American*, 223(4):120–123, 1970.
- Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination, 2020. URL <https://arxiv.org/abs/1912.01603>.
- Danijar Hafner, Wilson Yan, and Timothy Lillicrap. Training agents inside of scalable world models, 2025. URL <https://arxiv.org/abs/2509.24527>.
- Nicklas Hansen, Zhecheng Yuan, Yanjie Ze, Tongzhou Mu, Aravind Rajeswaran, Hao Su, Huazhe Xu, and Xiaolong Wang. On pre-training for visuo-motor control: Revisiting a learning-from-scratch baseline, 2023. URL <https://arxiv.org/abs/2212.05749>.
- Yixin Liu, Kai Zhang, Yuan Li, Zhiling Yan, Chujie Gao, Ruoxi Chen, Zhengqing Yuan, Yue Huang, Hanchi Sun, Jianfeng Gao, Lifang He, and Lichao Sun. Sora: A review on background, technology, limitations, and opportunities of large vision models, 2024. URL <https://arxiv.org/abs/2402.17177>.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization, 2019. URL <https://arxiv.org/abs/1711.05101>.
- Suraj Nair, Aravind Rajeswaran, Vikash Kumar, Chelsea Finn, and Abhinav Gupta. R3m: A universal visual representation for robot manipulation, 2022. URL <https://arxiv.org/abs/2203.12601>.
- Simone Parisi, Aravind Rajeswaran, Senthil Purushwalkam, and Abhinav Gupta. The unsurprising effectiveness of pre-trained vision models for control, 2022. URL <https://arxiv.org/abs/2203.03580>.
- Ilija Radosavovic, Tete Xiao, Stephen James, Pieter Abbeel, Jitendra Malik, and Trevor Darrell. Real-world robot learning with masked visual pre-training, 2022. URL <https://arxiv.org/abs/2210.03109>.
- Zhongwei Ren, Yunchao Wei, Xun Guo, Yao Zhao, Bingyi Kang, Jiashi Feng, and Xiaojie Jin. Videoworld: Exploring knowledge learning from unlabeled videos, 2025. URL <https://arxiv.org/abs/2501.09781>.
- Zhongwei Ren, Yunchao Wei, Xiao Yu, Guixun Luo, Yao Zhao, Bingyi Kang, Jiashi Feng, and Xiaojie Jin. Videoworld 2: Learning transferable knowledge from real-world videos, 2026. URL <https://arxiv.org/abs/2602.10102>.
- Moritz Schneider, Robert Krug, Narunas Vaskevicius, Luigi Palmieri, and Joschka Boedecker. The surprising ineffectiveness of pre-trained visual representations for model-based reinforcement learning, 2025. URL <https://arxiv.org/abs/2411.10175>.
- Stanford Research Computing Center. Sherlock Computing Cluster. <https://www.sherlock.stanford.edu>, 2014. Stanford University. RRID:SCR_026161.
- Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4):229–256, 1992.
- Hongtao Wu, Ya Jing, Chilam Cheang, Guangzeng Chen, Jiafeng Xu, Xinghang Li, Minghuan Liu, Hang Li, and Tao Kong. Unleashing large-scale video generative pre-training for visual robot manipulation, 2023. URL <https://arxiv.org/abs/2312.13139>.

A Compute Resources

All experiments were conducted on the Stanford Sherlock cluster [Stanford Research Computing Center, 2014], with each run allocated a single 80 GB GPU (NVIDIA H100 or A100) via SLURM using mixed-precision (bfloat16). Our full sweep comprises 128 independent training runs: 32 pretraining (4 fields $\times$ 4 layers $\times$ 2 weight decays), 64 pretrained RL (4 fields $\times$ 4 layers $\times$ 2 weight decays $\times$ 2 reward variants), and 32 from-scratch RL (4 fields $\times$ 4 layers $\times$ 2 reward variants). Pretraining time scales roughly linearly with model depth (~ 2 h for 1 layer to ~ 10 h for 4 layers), while RL wall-clock time is dominated by episode length, as agents in low-field environments survive more steps per episode. We estimate the total compute for the full sweep at approximately 1,100 GPU-hours.

B A Note on Reward Hacking

Initial Difficulties with RL. In our early experiments, we ran into reward hacking problems. When training from random initialization, the agent quickly learned to remain inside a safe corridor of cleared columns rather than exploring the live Game of Life field. Because the reward function provided positive reward for accurate GoL prediction at every step and only a small per-step penalty (-0.05), the agent could accumulate substantial return simply by surviving—up to $+0.9$ per step—over episodes lasting up to 300 steps. This resulted in increasing average return but zero progress toward the actual objective, with win rate and exploration both remaining near zero. This failure mode motivated the revised reward schemes (R_1 and R_2) described in Section 3, which tie reward directly to rightward progress rather than prediction accuracy.

C Supplemental Results



Figure 6: Pretrained vs. scratch comparison. Weight decay 10^{-3} , reward scheme R_1 .

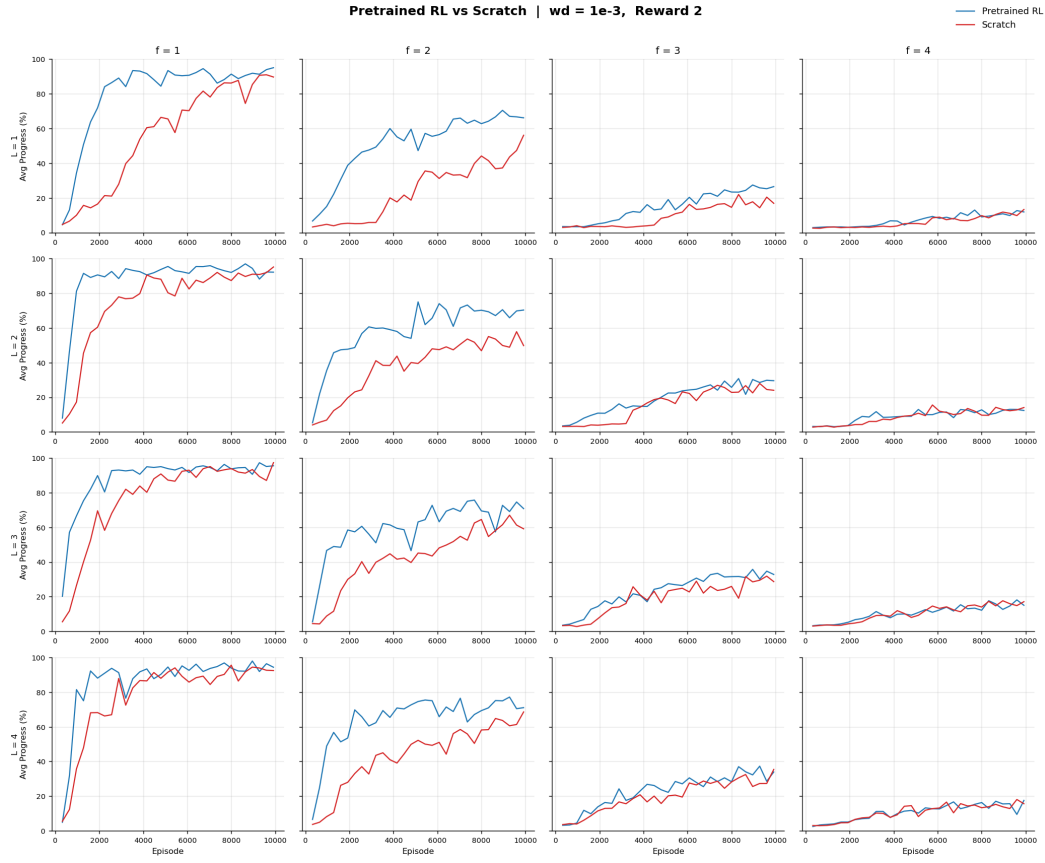


Figure 7: Pretrained vs. scratch comparison. Weight decay 10^{-3} , reward scheme R_2 .

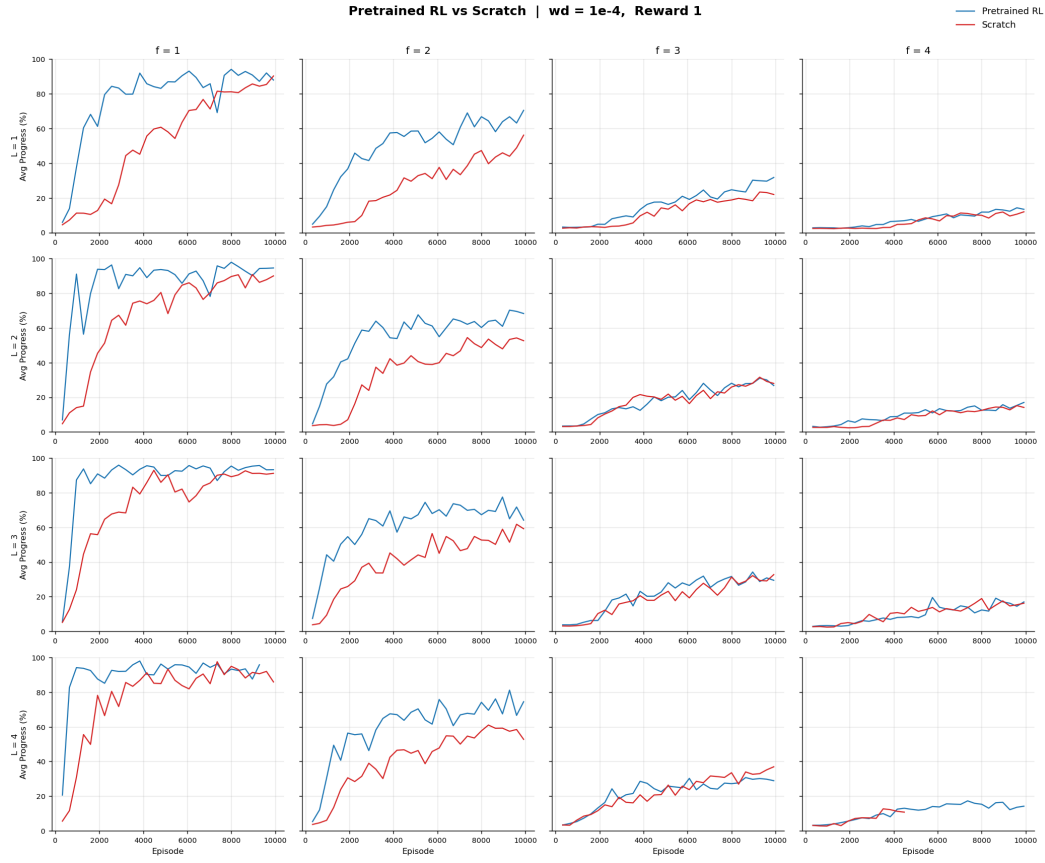


Figure 8: Pretrained vs. scratch comparison. Weight decay 10^{-4} , reward scheme R_1 .

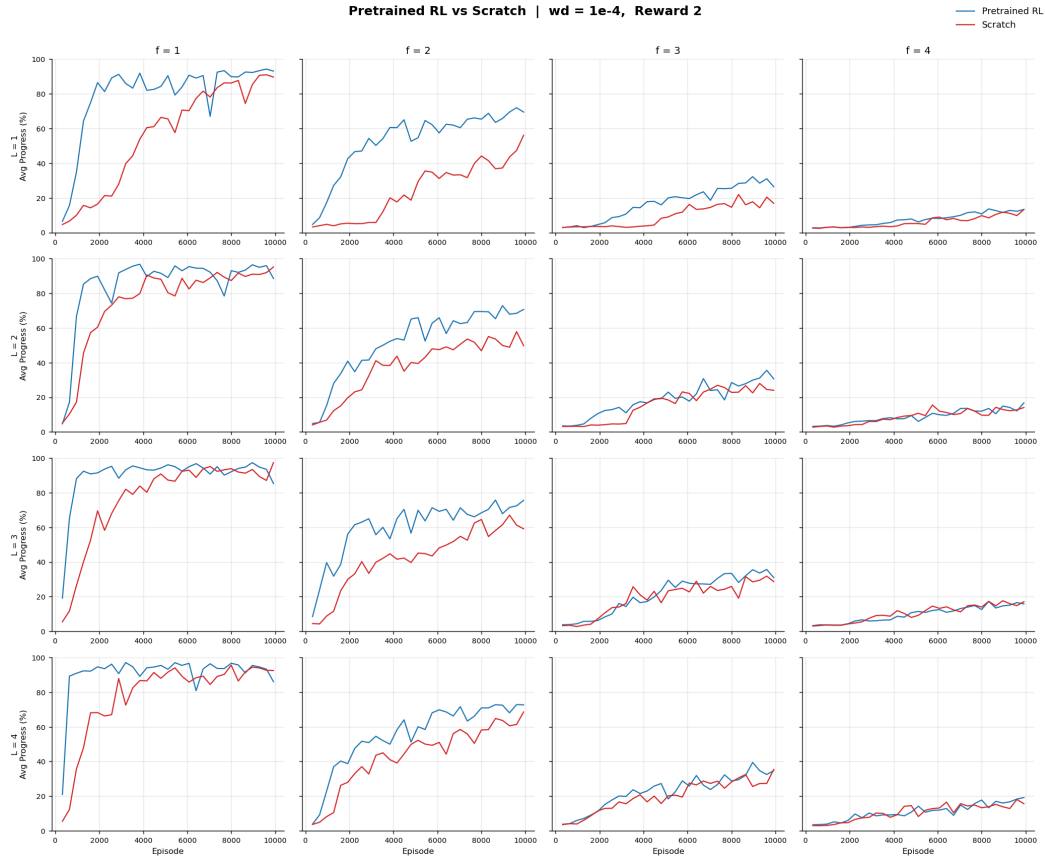


Figure 9: Pretrained vs. scratch comparison. Weight decay 10^{-4} , reward scheme R_2 .